

DOCUMENTO DE ENSAYO — v2

Seguridad como Código: DevSecOps Spec-Driven sobre GitHub para Banca

Techno Week 8.0 — Banco de Costa Rica — 18 de mayo 2026

Versión para simulación de capacitación — con interacciones Spark Up (sin demo en vivo)

Nota sobre esta versión

Este documento adapta la guía original del evento Techno Week para la sesión de simulación solicitada por los organizadores. El cambio principal: se sustituye la DEMO EN VIVO (17 min) por una **simulación guiada con 3 interacciones estratégicas de Spark Up**, manteniendo los mismos 48 minutos de contenido + 12 minutos de preguntas. Las interacciones no se colocan como adornos: cada una hace un trabajo narrativo específico y está ubicada donde el momento de la charla lo justifica.

Las secciones que mantienen el mismo contenido que la v1 aparecen resumidas; las secciones modificadas (especialmente la Sección 5) aparecen reescritas completas.

Estructura de la charla (48 min + 12 min preguntas)

#	SECCIÓN	DURACIÓN	ACUMULADO	SLIDES	SPARK UP
1	Apertura — El Hook	3:30	0:00–3:30	1–3	—
2	El Problema	6:30	3:30–10:00	4–8	★ Interacción 1
3	La Tesis	3:00	10:00–13:00	9–11	—
4	Arquitectura del Pipeline	7:00	13:00–20:00	12–17	—
5	Simulación guiada (reemplaza demo)	15:00	20:00–35:00	18–19	★ Interacción 2
6	Resultados e Impacto	3:00	35:00–38:00	20–23	—
7	GHAS + Estandarización	6:00	38:00–44:00	24–26	★ Interacción 3
8	ardops.dev + CTA	2:00	44:00–46:00	27–28	—
9	Cierre Memorable	2:00	46:00–48:00	29–30	—
10	Preguntas	12:00	48:00–60:00	—	—

★ = momento con interacción Spark Up. Tiempos de Spark Up incluidos dentro de la duración de cada sección (aprox. 60–75 seg por interacción: lanzar, esperar, leer resultados en vivo, comentar).

CHECKPOINTS: Min 10 = fin Problema + Interacción 1 | Min 13 = fin Tesis | Min 20 = inicio Simulación | Min 35 = fin Simulación | Min 38 = inicio GHAS + Interacción 3 | Min 48 = preguntas

SECCIÓN 1: APERTURA — EL HOOK

0:00 – 3:30 (3.5 min)

[SLIDE 1 — Portada] *[MOVER: Centro, mirar al público, NO a la pantalla] [TONO: Serio, pausado, como contando una historia real]*

El 14 de agosto del 2025, un atacante accedió a la API de una empresa financiera en Estados Unidos. No usó un exploit sofisticado. No rompió un firewall. Simplemente... hizo un request a un endpoint que no validaba quién estaba pidiendo los datos.

[PAUSA 3 seg]

En menos de 48 horas, los datos de 5.8 millones de personas estaban comprometidos. Nombres, números de seguro social, información crediticia. Todo.

[PAUSA 2 seg] [TONO: Conversacional]

Y saben qué es lo más absurdo? Esa vulnerabilidad — un endpoint sin validación de identidad — la hubiera detectado un linter de OpenAPI en 0.3 segundos.

[PAUSA 2 seg] [MOVER: Paso hacia el público] [TONO: Convicción]

0.3 segundos contra 5.8 millones de víctimas. Esa es la diferencia entre revisar la seguridad al final... y tenerla integrada desde el primer commit.

[SLIDE 2 — Nombre] *[TONO: Cálido, sonreír]*

Buenas tardes. Mi nombre es [TU NOMBRE], soy DevOps Engineer, y hoy vamos a hablar de cómo convertir la seguridad en código. No en documentos de 200 páginas. No en auditorías que llegan tres meses después. Código. Que se ejecuta, se versiona, y les dice en tiempo real si su API bancaria es segura o no.

[TONO: Ligeramente humorístico]

Y lo mejor: al final, cada uno va a tener acceso a un repositorio de GitHub donde todo esto funciona. Lo clonan, lo adaptan, y mañana ya tienen un pipeline de seguridad automatizado. Sin excusas.

[SLIDE 3 — ¿Qué tan segura es tu API?]

Pero antes, necesito que entendamos juntos por qué lo que hacemos hoy... no es suficiente.

SECCIÓN 2: EL PROBLEMA

3:30 – 10:00 (6.5 min) — incluye Interacción Spark Up #1

[SLIDE 4 — 84%]

En 2025, el 84% de las empresas reportaron al menos un incidente de seguridad en APIs. Y ojo — esos son los que lo detectaron.

★ INTERACCIÓN SPARK UP #1 — DIAGNÓSTICO DE AUDIENCIA (~70 seg)

Momento: 4:30 aprox — reemplaza el 'levanten la mano' original.

Pregunta: *¿En qué etapa se revisa seguridad en tu equipo?*

Opciones: (a) Antes del desarrollo · (b) Durante CI/CD · (c) Después (auditoría/pentest) · (d) No se revisa

Por qué aquí: te da un dato REAL de tu audiencia que vas a poder citar el resto de la charla. Si el grueso responde 'Después' o 'No se revisa' — que es lo estadísticamente probable — tenés munición narrativa para los siguientes 45 minutos.

[MIENTRAS RESPONDEN — 30 seg] [TONO: Conversacional, mirar pantalla de resultados]

Mientras llegan las respuestas... les cuento algo. Esta pregunta se la hice a 40 devs la semana pasada en un meetup. Prepárense para ver un número incómodo.

[CUANDO SE ESTABILIZA EL RESULTADO — leer en vivo] [TONO: Ajustar según lo que salga]

Si gana 'Después': "Ahí lo tienen. [X]% de ustedes está en el modelo del espejo retrovisor. Construir primero, revisar después. No les estoy contando una historia ajena — les estoy describiendo su realidad. Y eso es exactamente lo que vamos a cambiar en los próximos 40 minutos."

Si gana 'Durante CI/CD': "Buenas noticias — la mayoría ya tiene seguridad en CI/CD. Pero la pregunta no es SI la tienen, sino QUÉ tan temprano y QUÉ tan completa es. Vamos a ver si lo que están haciendo hoy es suficiente para SUGEF."

Si gana 'Antes del desarrollo': "Ok, tengo una audiencia madura. Eso cambia la conversación — hoy no les voy a vender la idea, les voy a mostrar cómo escalarla."

[SLIDE 5 — Solo el 13%]

Solo el 13% puede prevenir más de la mitad de los ataques. El otro 87%? En modo 'ojalá no nos pase'. Como poner un candado de bicicleta en la bóveda del banco.

[SLIDE 6 — Espejo retrovisor] El modelo clásico:

Desarrollo construye. Despliegan. Y DESPUÉS llega seguridad. Pen test. 47 hallazgos. PDF de 200 páginas. El dev lo abre y dice: 'Mae, esto lo veo la próxima semana.' Esa semana nunca llega.

[SLIDE 7 — Costa Rica SUGEF] [TONO: Serio, para tomadores de decisión]

En CR, CONASSIF reformó el Reglamento de Gestión TI en 2024. SUGEF exige controles verificables. Y en febrero 2026, la Asamblea aprobó en primer debate una ley que hace a los bancos responsables si no demuestran cumplimiento. Esto ya no es técnico. Es legal.

[SLIDE 8 — ¿Y si la spec fuera el contrato de seguridad?]

CHECKPOINT 10:00 — Fin Sección 2. Si atrasado por la interacción: saltar el ejemplo del 'candado de bicicleta'.

SECCIÓN 3: LA TESIS

10:00 – 13:00 (3 min) — SIN interacción (momento de autoridad del ponente)

Nota estratégica: Esta sección se mantiene sin interacción intencionalmente. Es tu momento "aha" — hablás desde el centro del escenario con convicción. Meter una encuesta aquí bajaría la intensidad. La propuesta original de colocar Spark Up en Tesis se descartó por esta razón.

[SLIDE 9 — Seguridad como Código] [MOVER: Centro] [TONO: Energético — momento aha!]

La solución no es más auditores ni más herramientas. Es cambiar CUÁNDO y CÓMO se aplica la seguridad. Qué pasaría si cada PR ejecutara 6 validaciones automáticas? Sin que nadie lo pida?

[SLIDE 10 — La Spec genera todo]

Spec-Driven Security: tu OpenAPI no es solo docs. Es el contrato de seguridad. Genera: linting, tests de auth, ZAP, deps, secretos, compliance. Todo en GitHub Actions. Todo como código.

[SLIDE 11 — Por qué Spec-Driven] Tres razones:

1) La spec es la fuente de verdad. 2) Es auditable — tu repo ES la evidencia para SUGEF. 3) Es reproducible — corre siempre igual.

[RESPIRAR]

CHECKPOINT 13:00 — Terminando tesis. Si atrasado, recortar una razón (dejar 1 y 2).

SECCIÓN 4: ARQUITECTURA DEL PIPELINE

13:00 – 20:00 (7 min)

[SLIDE 12 — Título] Ya entendemos el por qué. Ahora el cómo.

[SLIDE 13 — Pipeline] [MOVER: Señalar pantalla izq a der]

Todo empieza con openapi.yaml. Cuando un dev hace PR: 6 etapas automáticas.

[SLIDE 14 — Spectral + Semgrep]

Etapas 1: Spectral — lint de la spec. Etapa 2: Semgrep — SAST, patrones peligrosos.

[SLIDE 15 — Gitleaks + npm audit]

Etapas 3: Gitleaks — secretos. (Broma: todos hemos sudado frío con un git push) Etapa 4: npm audit — CVEs.

[SLIDE 16 — OWASP ZAP + Compliance]

Etapas 5: ZAP — levanta API, la ataca con la spec. Etapa 6: reporte automático en el PR.

[SLIDE 17 — Vista del PR]

Verde = seguro. Rojo = corregir. El reviewer no necesita ser experto en seguridad.

[TONO: Confiado, transición]

Y ahora... en lugar de solo contarles cómo funciona... vamos a **vivirlo juntos**. Ustedes van a ser parte del análisis.

CHECKPOINT 20:00 — Empezando Simulación. Si atrasado, recortar la Etapa 4 (npm audit).

SECCIÓN 5: SIMULACIÓN GUIADA — REEMPLAZO DE DEMO

20:00 – 35:00 (15 min) — incluye Interacción Spark Up #2

Sección reescrita. Esta sección sustituye la DEMO EN VIVO de la v1 (17 min). La reducción a 15 min es intencional: sin ejecución real, estirla más de 15 min pierde tensión. Los 2 minutos liberados se reparten entre la Interacción 1 (Sección 2, +1 min) y la Sección 7 GHAS (+1 min).

[SLIDE 18 — 'Vamos a simular un PR vulnerable'] [MOVER: Caminar un paso hacia la audiencia, NO hacia la laptop]

[TONO: Conspirativo, bajar un poco la voz]

No voy a hacer demo en vivo hoy. Les voy a ser honesto — y esto tiene una razón: **las demos en vivo son frágiles**. Y lo más importante de este pipeline no es verlo correr una vez. Es que ustedes entiendan **cómo razona**. Así que vamos a hacer algo mejor: vamos a simular juntos un Pull Request vulnerable, paso por paso, y ustedes me van a decir qué creen que va a pasar.

Flujo de la simulación (15 min)

TIEMPO	PASO	QUÉ HACER / DECIR
20:00–22:00	1. Contexto	[SLIDE 18] Mostrar screenshot del repo: README, estructura, SECURITY.md. Describir brevemente
22:00–24:30	2. La spec	[SLIDE con snippet OpenAPI] 3 endpoints sanos. Señalar el campo security en cada uno. Formato
24:30–26:00	3. La vulnerabilidad	[TONO: Conspirativo] 'Ahora imaginen que un dev agrega este endpoint nuevo...' Mostrar snippet
26:00–28:30	4. SPARK UP #2	★ Lanzar interacción: '¿Cuál de los 6 checks creen que lo va a detectar primero?' (ver detalle aba
28:30–32:00	5. El resultado	[Screenshots del pipeline en rojo] Spectral ROJO primero. Luego Semgrep ROJO. Luego ZAP ROJO
32:00–33:30	6. El reporte	[Screenshot del comentario del bot en el PR] Semáforo visual. Explicar qué ve el reviewer.
33:30–35:00	7. El fix	[Screenshot del diff con security agregado] [Screenshot del pipeline VERDE] 'Listo para merge.'

★ INTERACCIÓN SPARK UP #2 — SIMULACIÓN (momento 26:00, ~2.5 min total)

Setup (30 seg): "Lo que acaban de ver es un endpoint sin el campo 'security'. En nuestro pipeline corren 6 checks en paralelo. Ustedes son el reviewer. ¿Cuál creen que lo detecta PRIMERO?"

Pregunta: ¿Cuál check va a detectar la vulnerabilidad primero?

Opciones: (a) Spectral (spec lint) · (b) Semgrep (SAST) · (c) OWASP ZAP (DAST) · (d) Los tres lo detectan

Mientras votan (30 seg): "Piénsenlo bien. Cada herramienta ve algo distinto. Spectral mira la spec. Semgrep mira el código. ZAP ataca la API corriendo. Solo una de las tres lo ve ANTES de que se levante el servicio."

Revelación (90 seg): La respuesta correcta es **(d) Los tres lo detectan**, pero el ORDEN importa:

- **Spectral** lo ve en **milisegundos** — la spec no tiene el campo 'security', la regla falla.
- **Semgrep** lo ve en **segundos** — encuentra el handler del endpoint sin middleware de auth.
- **ZAP** lo ve en **minutos** — ataca el endpoint en runtime y confirma la ausencia de 401.

Punch line: "Esta es la belleza del spec-driven: la detección más rápida y más barata ocurre antes de compilar, antes de desplegar, antes de gastar un segundo de CPU. 0.3 segundos. ¿Recuerdan el número del inicio?"

[Después del Paso 7, volver al centro del escenario]

[SLIDE 19 — Recapitulación visual con los 3 semáforos: ANTES → DURANTE → DESPUÉS del fix] "0.3 segundos vs 5.8 millones. Recuerdan?"

PLAN B si Spark Up falla técnicamente:

(1) Hacer la pregunta a mano alzada — la narrativa no cambia, solo pierde el dato cuantitativo. (2) Saltar a los screenshots y decir: *"Mientras los organizadores revisan, déjenme mostrarles directamente lo que habría pasado."* NUNCA disculparse con 'esto funcionaba hace 5 minutos'.

CHECKPOINT 35:00 — Fin Simulación. Volver al centro. Respirar. Tomar agua.

SECCIÓN 6: RESULTADOS E IMPACTO

35:00 – 38:00 (3 min)

[SLIDE 20 — Resultados] + [SLIDE 21 — Antes vs Después]

Detección: de semanas a minutos. Cobertura: de 60% manual a 100% de la spec. Evidencia: de PDFs perdidos a trazabilidad en cada PR.

[SLIDE 22 — \$9.36 millones]

Costo promedio de un breach en banca en EE.UU. Implementar este pipeline cuesta: tiempo, herramientas open source, y voluntad.

[SLIDE 23 — Compliance automático] [TONO: Para tomadores de decisión]

Cada PR genera evidencia. Cada merge tiene trazabilidad. Cuando SUGEF llegue, la evidencia ya existe.

SECCIÓN 7: GHAS + ESTANDARIZACIÓN ★

38:00 – 44:00 (6 min) — incluye Interacción Spark Up #3

★ Sección de gerencia — cumple con el requisito sobre GitHub Advanced Security y estandarización organizacional. Se expande +1 min respecto a v1 para dar espacio a la Interacción 3.

[SLIDE 24 — GitHub Advanced Security] [TONO: Conectando con realidad enterprise]

Todo lo que mostré usa herramientas open source. Pero GitHub ofrece algo nativo que para un banco es game changer: GitHub Advanced Security. Tres componentes:

Primero: CodeQL. Motor de análisis semántico de GitHub. No solo busca patrones como Semgrep — entiende cómo los datos fluyen desde el input del usuario hasta la base de datos. Se configura con dos clicks o con un workflow de Actions.

Segundo: Secret Scanning con Push Protection. Clave para banca. En vez de detectar un secreto DESPUÉS del push, GitHub lo bloquea ANTES. El dev intenta hacer push con una API key y GitHub dice: "No. Esto no sale de tu máquina." Prevención, no solo detección.

Tercero: Dependabot. Monitorea dependencias 24/7. Crea PRs automáticos cuando hay un CVE. No hay que esperar a que alguien corra npm audit.

★ INTERACCIÓN SPARK UP #3 — DECISIÓN ORGANIZACIONAL (~75 seg)

Momento: 40:30 aprox — antes del SLIDE 25 (modelo híbrido).

Pregunta: Para su organización hoy, ¿qué es más valioso?

Opciones: (a) Prevención · (b) Detección · (c) Cumplimiento (compliance) · (d) Auditoría

Por qué aquí: personalizás el cierre técnico según lo que predomine, y sembrás la conversación para las preguntas del Q&A. Cada opción te da un ángulo distinto para conectar con GHAS.

[Leer resultados en vivo y ajustar el énfasis del cierre]

Si gana Prevención: "Entonces Push Protection es su inversión número uno. Es la única herramienta que bloquea ANTES de que el problema exista."

Si gana Detección: "CodeQL es donde empiezan. Análisis semántico profundo, integrado en cada PR, con visibilidad en toda la organización."

Si gana Cumplimiento: "Para eso la combinación open source + GHAS es perfecta. Cada ejecución queda registrada en GitHub. Cada política es código. SUGEF puede auditar el repo directamente."

Si gana Auditoría: "Organization Rulesets + Custom Properties. Desde un solo lugar ven qué repos cumplen qué política, y qué excepciones existen. Evidencia instantánea."

[SLIDE 25 — Open Source + GHAS: modelo híbrido]

Mi recomendación para un banco: modelo híbrido. Las herramientas open source del pipeline — Spectral, ZAP — son el core para seguridad spec-driven. GHAS agrega la capa enterprise: CodeQL para análisis profundo, Push Protection para prevención, Dependabot para monitoreo continuo. Secret Protection: \$19/commiter/mes. Code Security: \$30. Para repos públicos, muchas features son gratis.

[SLIDE 26 — De 1 repo a 200] [TONO: Dirigido a tomadores de decisión]

La pregunta de gerencia: esto funciona para un repo, ¿cómo lo aplico a los 200 del banco? Tres mecanismos:

Uno: Reusable Workflows. Definir el pipeline UNA vez en un repo central. Todos los equipos lo invocan con una sola línea. Si actualizás una regla, se propaga a todos.

Dos: Organization RuleSets. Desde la config de la org, reglas obligatorias: ningún PR se mergea a main sin que el workflow de seguridad pase. Ni un admin puede saltárselo sin quedar registrado.

Tres: Custom Properties. Clasificar repos por nivel de riesgo — alto, medio, bajo. Repos con datos de clientes: 6 etapas obligatorias. Repos internos: subconjunto.

Resultado: un estándar de seguridad organizacional, definido como código, aplicado automáticamente, auditado por GitHub. Exactamente lo que SUGEF y la nueva ley necesitan ver.

CHECKPOINT 44:00 — Terminando GHAS. Transición a ardops.dev.

SECCIONES 8–9: ARDOPS.DEV + CIERRE

44:00 – 48:00 (4 min)

[SLIDE 27 — *ardops.dev*] + [SLIDE 28 — QR]

Todo lo que vimos hoy está en *ardops.dev*. Repo, slides, guías. [QR] Sáquenle foto ahora. [PAUSA 10 seg]

[SLIDE 29 — *La seguridad no es un gate. Es un guardrail.*]

[MOVER: Centro. Contacto visual. TONO: Pausado, con peso]

La seguridad no es un gate. No es una barrera al final. Es un guardrail. Está ahí MIENTRAS construyes. Te guía. Te protege sin frenarte.

[PAUSA 3 seg]

Mi reto: la próxima vez que hagan un PR, pregúntense: ¿cuántos checks de seguridad se ejecutaron automáticamente? Si la respuesta es cero... ya saben por dónde empezar.

[SLIDE 30 — *Gracias / ardops.dev*] [PAUSA 3 seg]

Gracias. El repo es público, *ardops.dev* tiene todo. Estoy aquí para preguntas.

[MOVER: Quedarse en el centro. Tomar agua. No ir al podio.]

CHECKPOINT 48:00 — Abriendo preguntas. Si nadie pregunta arrancar con planted question: 'Una pregunta que siempre me hacen: ¿esto no hace más lento el desarrollo?'

REFERENCIA RÁPIDA

Resumen de Interacciones Spark Up

#	MOMENTO	PREGUNTA	FUNCIÓN NARRATIVA
1	4:30 (Sección 2)	¿En qué etapa se revisa seguridad en tu equipo?	Demuestra de audiencia — dato citable el resto de la charla
2	26:00 (Sección 5)	¿Cuál check detecta la vulnerabilidad principal?	Reemplaza la demo — convierte audiencia en analista
3	40:30 (Sección 7)	¿Qué es más valioso para su organización?	Personaliza cierre GHAS — siembra Q&A

Datos Clave

- 84% de empresas tuvieron incidente en APIs (Traceable.ai 2025)
- Solo 13% puede prevenir más del 50% de ataques a APIs
- \$9.36M costo promedio de breach en banca EE.UU. (2024, Statista/IBM)
- 5.8M registros en breach de 700Credit (ago 2025, API sin validación)
- 3,322 data breaches en EE.UU. en 2025 — récord histórico (ITRC)
- 57% de orgs sufrieron breach por APIs en últimos 2 años
- CONASSIF 5-24: reforma Reglamento Gestión TI (julio 2024)
- Ley de responsabilidad bancaria: primer debate, febrero 2026
- GHAS: Secret Protection \$19/mes, Code Security \$30/mes por committer

Pipeline (7 etapas con GHAS)

#	HERRAMIENTA	TIPO	QUÉ HACE
1	Spectral	Spec Lint	Reglas de seguridad en OpenAPI
2	Semgrep	SAST	Patrones de código peligrosos
3	Gitleaks	Secrets	API keys, passwords en código
4	npm audit	Deps	CVEs en dependencias
5	OWASP ZAP	DAST	Ataca la API con la spec
6	Custom Action	Report	Reporte compliance en PR
7	CodeQL (GHAS)	SAST+	Análisis semántico de flujo de datos

Estandarización Organizacional

- **Reusable Workflows:** definir UNA vez, invocar desde N repos con 1 línea
- **Organization Rulesets:** ningún PR se mergea sin pasar seguridad (obligatorio)
- **Custom Properties:** clasificar repos por riesgo (alto/medio/bajo)

Frases de Emergencia

No sabes la respuesta: "Excelente pregunta. ¿Me comparten su contacto? Les mando la respuesta esta semana."

Te interrumpen: "Entiendo. Permítame terminar y lo discutimos en preguntas."

Spark Up falla: "Hagámoslo a mano alzada entonces — a veces lo simple es mejor."

Te quedás en blanco: Ir a la slide. Leer título. Tomar agua. "Déjenme retomar..."

Preguntan por costo GHAS: "\$19 Secret Protection + \$30 Code Security por committer/mes. Repos públicos:

gratis."

Checklist pre-simulación (verificar con organizadores BCR)

- Confirmar nombre y plataforma exacta de Spark Up (puede variar entre tools internas)
- Tiempo de respuesta desde lanzar pregunta hasta ver resultados (ajusta los 70 seg estimados)
- ¿Resultados visibles en vivo o solo al cerrar votación?
- ¿Se puede proyectar simultáneamente la pregunta y las slides o hay que alternar?
- Prueba técnica 30 min antes del evento — lanzar pregunta dummy
- Plan B: mano alzada + contador mental si Spark Up falla

Hablar al 70% de velocidad. Pausar después de datos impactantes. Contacto visual 3 zonas. Manos fuera de bolsillos. Agua en transiciones. Respirar. Sonreír.